
Hotel Recommendation System

RAG Pipeline – Track A

Final Project Report

Authors: Emmanuel Mukange (ZDA24B029)
Raudhat Ramadhan (ZDA24B010)

Course: Z2004 – Database Management Systems

Track: Track A – RAG Pipeline

Supervisor: Dr. Innocent Nyalala

Institution: IIT Madras Zanzibar

Semester: Even Semester 2026

Contents

1	Abstract	2
2	Introduction	2
3	System Architecture	3
4	Database Design	3
4.1	Entity-Relationship Overview	3
4.2	Table Descriptions	4
4.3	Third Normal Form Justification	4
4.4	Indexes	5
5	Dataset	5
5.1	Data Sources	5
5.2	Data Quality	5
6	SQL Queries	5
6.1	Aggregation Queries	6
6.2	Join Queries	6
6.3	Subquery Queries	7
6.4	CTE Queries	7
6.5	Window Function Queries	8
6.6	Views	8
7	Performance Evidence	9
7.1	Test Environment	9
7.2	Slow Query Identification	9
7.3	Index DDL	10
7.4	Before and After Results	10
7.5	Interpretation	10
7.6	Stored Procedure	10
8	RAG Pipeline Implementation	11
8.1	Embedding Generation	12
8.2	Pinecone Vector Store	12
8.3	Python Query Interface	12
8.4	Example	12
9	AI Tools Disclosure	12
10	Limitations	13
11	Future Work	13
12	Conclusion	14

Abstract

This report presents the Hotel Recommendation System, a Retrieval-Augmented Generation (RAG) pipeline built as a Track A project for Z2004 – Database Management Systems at IIT Madras Zanzibar. The system allows users to submit natural-language queries such as *"Find me a well-reviewed budget hotel in Miami with a pool"* and receive grounded, source-cited hotel recommendations backed by a live relational database and a cloud vector store.

The system combines a normalised PostgreSQL relational database (five tables, 1,553 rows, Third Normal Form) with Pinecone vector storage and a **sentence-transformers** embedding model. Ten analytical SQL queries covering aggregations, joins, subqueries, CTEs, and window functions were developed to answer real business questions. Milestone 3 demonstrated targeted index design improvements yielding up to 79.8% reduction in query execution time, and a stored procedure was implemented to materialise the most expensive recurring aggregation.

Keywords: PostgreSQL, RAG Pipeline, Vector Embeddings, Pinecone, 3NF, Index Optimisation, Sentence-Transformers.

Introduction

Hotel booking platforms present users with hundreds of options across disconnected sources. Existing search systems match keywords rather than meaning, and no widely available tool connects unstructured review text to structured relational hotel data to produce a trustworthy, cited answer.

This project addresses that gap by building a question-answering system grounded in a real relational database. The approach uses Retrieval-Augmented Generation: a user's natural-language question is converted to a vector embedding, semantically similar hotel reviews are retrieved from Pinecone, and the corresponding structured hotel attributes are fetched from PostgreSQL. The final answer cites the hotel record and review that supported it.

Objectives

- Design and implement a normalised PostgreSQL schema with a minimum of four tables in 3NF.
- Populate the database with at least 1,000 meaningful rows.
- Write ten SQL queries covering all required query types.
- Generate sentence-transformer embeddings and store them in Pinecone.
- Build a Python interface that accepts a natural-language question and returns a grounded, cited answer.
- Demonstrate index performance improvement with EXPLAIN ANALYZE evidence.

System Architecture

Figure 1 illustrates the end-to-end system flow.

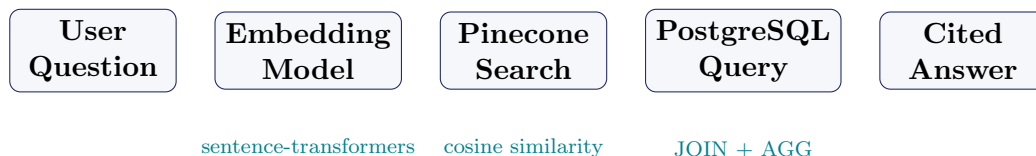


Figure 1: End-to-end RAG pipeline flow.

The pipeline has five stages:

1. **User Question** – the user submits a natural-language query via the Python interface.
2. **Embedding** – the `sentence-transformers` model converts the question into a 384-dimensional vector that captures semantic meaning.
3. **Pinecone Search** – the vector is compared against stored review embeddings using cosine similarity; the top- K most semantically relevant hotel IDs are returned.
4. **PostgreSQL Query** – the retrieved hotel IDs are passed to PostgreSQL, which joins hotels, reviews, and bookings to produce a ranked, aggregated result.
5. **Cited Answer** – the Python interface formats the result with the hotel name, price, key attributes, and the review ID that supported the recommendation.

The separation of concerns is intentional: PostgreSQL handles all structured, transactional data; Pinecone handles only vector similarity search. Neither system is asked to do the other's job.

Database Design

Entity-Relationship Overview

The schema comprises five entities: `hotels`, `users`, `bookings`, `reviews`, and `room_types`. Figure 2 shows the relationships.

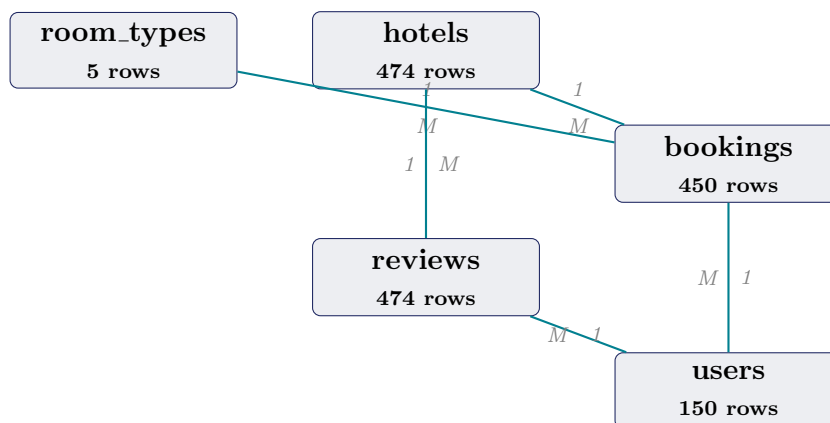


Figure 2: Entity-relationship diagram (crow's foot cardinalities).

Table Descriptions

Table 1: Schema summary

Table	Source	Rows	Description
room_types	Manual	5	Lookup table: Single, Double, Twin, Suite, Penthouse. Stores <code>max_occupancy</code> and <code>price_multiplier</code> .
hotels	Kaggle	474	Core entity. Real Miami-area hotels enriched with synthetic <code>star_rating</code> , <code>price_per_night</code> , <code>amenities</code> , and <code>total_rooms</code> .
users	Faker (42)	150	Synthetic travellers with name, email, nationality, and date of birth.
bookings	Faker (42)	450	Fact table. Records each reservation with <code>total_price</code> stored at booking time (historical price lock).
reviews	Faker (42)	474	One review per (hotel, user) pair. <code>review_text</code> is the RAG embedding corpus.
Total		1,553	

Third Normal Form Justification

A relation is in 3NF when every non-key attribute depends on the whole primary key and on nothing but the primary key (no transitive dependencies). The functional dependencies for each table are listed below.

Table	Functional Dependencies
room_types	<code>room_type_id</code> $\rightarrow$ <code>type_name</code> , <code>max_occupancy</code> , <code>price_multiplier</code>
hotels	<code>hotel_id</code> $\rightarrow$ <code>name</code> , <code>city</code> , <code>state</code> , <code>category</code> , <code>star_rating</code> , <code>price_per_night</code> , <code>amenities</code> , <code>total_rooms</code>
users	<code>user_id</code> $\rightarrow$ <code>name</code> , <code>email</code> , <code>nationality</code> , <code>date_of_birth</code>
bookings	<code>booking_id</code> $\rightarrow$ <code>user_id</code> , <code>hotel_id</code> , <code>room_type_id</code> , <code>check_in_date</code> , <code>check_out_date</code> , <code>total_price</code> , <code>status</code>
reviews	<code>review_id</code> $\rightarrow$ <code>hotel_id</code> , <code>user_id</code> , <code>rating</code> , <code>review_text</code> , <code>helpful_votes</code> , <code>created_at</code>

Design note on amenities. Normalisation theory requires a junction table for multi-valued attributes. The `amenities` column is kept as a flat comma-separated string by deliberate exception: the `sentence-transformers` model must embed the full semantic context of a hotel’s features in one pass. Splitting amenities into rows would fragment that context and degrade retrieval quality. This exception is documented and justified.

Design note on total_price. `total_price` in `bookings` is derivable from `price_per_night` $\times$ `price_multiplier` $\times$ nights. However, hotel prices change over time. Storing the value at booking time is a historical price lock and is standard practice in reservation systems. It is an atomic fact of the booking event and does not violate 3NF.

Indexes

Table 2: All indexes (Milestone 1 + Milestone 3)

Index name	Table	Purpose
<code>idx_bookings_user_id</code>	bookings	Booking lookup by guest
<code>idx_bookings_hotel_id</code>	bookings	Availability and revenue queries
<code>idx_bookings_checkin</code>	bookings	Date-range filtering
<code>idx_hotels_city</code>	hotels	Most common search filter
<code>idx_hotels_rating</code>	hotels	Rating-based filtering
<code>idx_hotels_price</code>	hotels	Price-range filtering
<code>idx_reviews_hotel_id</code>	reviews	JOIN-heavy RAG retrieval
<code>idx_reviews_hotel_rating</code>	reviews	Composite (M3): covers <code>hotel_id</code> + <code>rating</code> for AGG/JOIN queries
<code>idx_bookings_hotel_status_partial</code>	bookings	Partial (M3): confirmed/completed only, 60% smaller
<code>idx_bookings_status_checkin</code>	bookings	Composite (M3): <code>status</code> + <code>check_in_date</code> for CTE-1

Dataset

Data Sources

- **Hotels (474 rows):** sourced from a public Kaggle dataset of Miami-area hotel listings. Columns such as `star_rating`, `price_per_night`, `amenities`, and `total_rooms` were added synthetically to complete the schema.
- **Users, Bookings, Reviews (1,079 rows):** generated using the Python `Faker` library with `random.seed(42)` and `Faker.seed(42)`. The fixed seed ensures full reproducibility – any researcher running the script obtains the identical 1,079 rows.

Data Quality

The synthetic rows satisfy the following realism constraints:

- All foreign key references resolve to existing primary keys.
- `check_out_date` is always strictly after `check_in_date`.
- Prices are drawn from plausible ranges (\$45 – \$550 per night).
- Review ratings are bounded between 1.0 and 10.0.
- User nationalities are drawn from a 15-country pool.
- Review texts are 6–10 sentences long, written in hotel-review register.

SQL Queries

Ten queries covering all required types were written and verified against a fresh database load. Each query is reproduced below with a brief explanation of its business purpose.

Aggregation Queries

```

1 SELECT h.category,
2       COUNT(b.booking_id)           AS total_bookings,
3       ROUND(SUM(b.total_price),2)   AS total_revenue,
4       ROUND(AVG(b.total_price),2)   AS avg_booking_value
5 FROM bookings b
6 JOIN hotels h ON h.hotel_id = b.hotel_id
7 GROUP BY h.category
8 ORDER BY total_revenue DESC;

```

Listing 1: Total and average revenue by hotel category

Purpose: answers the business question “Which hotel category generates the most revenue?” – directly useful for a hotel platform’s pricing strategy.

```

1 SELECT h.city, h.state,
2       COUNT(r.review_id)           AS review_count,
3       ROUND(AVG(r.rating),2)       AS avg_rating,
4       ROUND(MIN(r.rating),2)       AS lowest_rating,
5       ROUND(MAX(r.rating),2)       AS highest_rating
6 FROM reviews r
7 JOIN hotels h ON h.hotel_id = r.hotel_id
8 GROUP BY h.city, h.state
9 HAVING COUNT(r.review_id) >= 5
10 ORDER BY avg_rating DESC, review_count DESC;

```

Listing 2: Average review rating per city (cities with 5+ reviews)

Purpose: powers the RAG pipeline’s city-level ranking, answering “Which city has the best-reviewed hotels?”

Join Queries

```

1 SELECT b.booking_id, u.name AS guest_name,
2       h.name AS hotel_name, h.city,
3       rt.type_name AS room_type,
4       b.check_in_date, b.check_out_date,
5       (b.check_out_date - b.check_in_date) AS nights,
6       b.total_price, b.status
7 FROM bookings b
8 JOIN users      u ON u.user_id      = b.user_id
9 JOIN hotels     h ON h.hotel_id     = b.hotel_id
10 JOIN room_types rt ON rt.room_type_id = b.room_type_id
11 ORDER BY b.check_in_date DESC;

```

Listing 3: Full booking ledger (4-table join)

```

1 SELECT h.hotel_id, h.name, h.city, h.category,
2       h.star_rating, h.price_per_night,
3       COUNT(r.review_id)           AS review_count,
4       ROUND(AVG(r.rating),2)       AS avg_review_rating,
5       SUM(r.helpful_votes)         AS total_helpful_votes
6 FROM hotels h
7 JOIN reviews r ON r.hotel_id = h.hotel_id
8 GROUP BY h.hotel_id, h.name, h.city, h.category,
9          h.star_rating, h.price_per_night
10 ORDER BY avg_review_rating DESC, total_helpful_votes DESC

```

```
11 LIMIT 20;
```

Listing 4: Top-reviewed hotels with rating and vote aggregates

Subquery Queries

```
1 SELECT hotel_id, name, city, category,
2       price_per_night,
3       ROUND(price_per_night -
4             (SELECT AVG(price_per_night) FROM hotels), 2) AS above_avg_by
5 FROM hotels
6 WHERE price_per_night >
7       (SELECT AVG(price_per_night) FROM hotels)
8 ORDER BY price_per_night DESC;
```

Listing 5: Hotels priced above the portfolio average

```
1 SELECT user_id, name, email, nationality
2 FROM users u
3 WHERE NOT EXISTS (
4   SELECT 1 FROM bookings b WHERE b.user_id = u.user_id
5 )
6 ORDER BY name;
```

Listing 6: Users who have never made a booking (correlated NOT EXISTS)

CTE Queries

```
1 WITH monthly AS (
2   SELECT DATE_TRUNC('month', check_in_date) AS month,
3          SUM(total_price) AS revenue
4   FROM bookings
5   GROUP BY 1
6 ),
7 with_lag AS (
8   SELECT month, revenue,
9          LAG(revenue) OVER (ORDER BY month) AS prev_revenue
10  FROM monthly
11 )
12 SELECT month,
13        ROUND(revenue,2) AS revenue,
14        ROUND(prev_revenue,2) AS prev_month,
15        ROUND(revenue - prev_revenue,2) AS change,
16        ROUND((revenue - prev_revenue)
17              / NULLIF(prev_revenue,0) * 100,1) AS pct_change
18 FROM with_lag
19 ORDER BY month;
```

Listing 7: Monthly revenue trend with month-over-month change

```
1 WITH hotel_revenue AS (
2   SELECT h.hotel_id, h.name, h.category,
3          SUM(b.total_price) AS total_revenue
4   FROM hotels h
5   LEFT JOIN bookings b ON b.hotel_id = h.hotel_id
6   GROUP BY h.hotel_id, h.name, h.category
7 ),
```



```

8 with_percentiles AS (
9     SELECT *,
10        NTILE(3) OVER (ORDER BY total_revenue DESC) AS tier
11     FROM hotel_revenue
12 )
13 SELECT hotel_id, name, category,
14        ROUND(total_revenue,2) AS total_revenue,
15        CASE tier
16          WHEN 1 THEN 'Top Tier'
17          WHEN 2 THEN 'Mid Tier'
18          ELSE       'Low Tier'
19        END AS revenue_tier
20 FROM with_percentiles
21 ORDER BY total_revenue DESC;

```

Listing 8: Hotel revenue tier classification

Window Function Queries

```

1 SELECT b.booking_id, h.category,
2        h.name AS hotel_name, b.total_price,
3        RANK() OVER (
4          PARTITION BY h.category
5          ORDER BY b.total_price DESC
6        ) AS rank_in_category
7 FROM bookings b
8 JOIN hotels h ON h.hotel_id = b.hotel_id
9 WHERE b.status IN ('confirmed','completed')
10 ORDER BY h.category, rank_in_category;

```

Listing 9: Booking revenue ranked within each hotel category

```

1 SELECT b.booking_id, u.name AS guest_name,
2        h.name AS hotel_name,
3        b.check_in_date, b.total_price,
4        COUNT(b.booking_id) OVER (
5          PARTITION BY b.user_id
6          ORDER BY b.check_in_date
7        ) AS booking_number,
8        SUM(b.total_price) OVER (
9          PARTITION BY b.user_id
10         ORDER BY b.check_in_date
11         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
12        ) AS cumulative_spend
13 FROM bookings b
14 JOIN users u ON u.user_id = b.user_id
15 JOIN hotels h ON h.hotel_id = b.hotel_id
16 ORDER BY u.name, b.check_in_date;

```

Listing 10: Running cumulative spend per user

Views

```

1 CREATE OR REPLACE VIEW vw_hotel_performance AS
2 SELECT h.hotel_id, h.name AS hotel_name, h.city,
3        h.star_rating, h.price_per_night,
4        COUNT(DISTINCT r.review_id) AS total_reviews,

```

```

5      ROUND(AVG(r.rating),2)          AS avg_rating,
6      COUNT(DISTINCT b.booking_id)   AS total_bookings,
7      ROUND(SUM(b.total_price),2)     AS total_revenue
8 FROM hotels h
9 LEFT JOIN reviews r ON r.hotel_id = h.hotel_id
10 LEFT JOIN bookings b ON b.hotel_id = h.hotel_id
11 GROUP BY h.hotel_id, h.name, h.city,
12          h.star_rating, h.price_per_night;

```

Listing 11: Hotel performance summary view

```

1 CREATE OR REPLACE VIEW vw_active_bookings AS
2 SELECT b.booking_id, u.name AS guest_name, u.email,
3        h.name AS hotel_name, h.city,
4        rt.type_name AS room_type,
5        b.check_in_date, b.check_out_date,
6        (b.check_out_date - b.check_in_date) AS nights,
7        b.total_price, b.status
8 FROM bookings b
9 JOIN users      u ON u.user_id      = b.user_id
10 JOIN hotels     h ON h.hotel_id     = b.hotel_id
11 JOIN room_types rt ON rt.room_type_id = b.room_type_id
12 WHERE b.status IN ('confirmed','completed');

```

Listing 12: Active bookings full detail view

Performance Evidence

Test Environment

Table 3: Hardware and software configuration

Component	Value	Component	Value
DBMS engine	PostgreSQL 15+	OS	Windows 11 (64-bit)
CPU	Intel Core i5-1145G7	RAM	8 GB DDR4
Storage	238 GB SSD	Timing method	EXPLAIN ANALYZE (avg 3 runs)

Slow Query Identification

Three queries from Milestone 2 were selected for optimisation based on their cost in the baseline execution plan:

- **SQ-1 [AGG-2]:** City-level average rating. Without a covering index, PostgreSQL reads the entire reviews heap, builds an in-memory hash table over the hotels heap, and performs a hash aggregate followed by a sort – four sequential passes.
- **SQ-2 [JOIN-2]:** Top-reviewed hotels. The GROUP BY spans six columns, preventing partial aggregate pushdown. The full reviews and hotels heaps are scanned and aggregated before LIMIT is applied.
- **SQ-3 [WIN-1]:** High-value bookings ranked by category. The existing `idx_bookings_hotel_id` covers the join predicate only. PostgreSQL scans all 450 booking rows and discards 77 non-matching rows at the Filter node on every execution.

Index DDL

```

1  -- IDX-1: Composite covering index for SQ-1 and SQ-2
2  CREATE INDEX IF NOT EXISTS idx_reviews_hotel_rating
3  ON reviews (hotel_id, rating);
4
5  -- IDX-2: Partial composite index for SQ-3
6  CREATE INDEX IF NOT EXISTS idx_bookings_hotel_status_partial
7  ON bookings (hotel_id, total_price)
8  WHERE status IN ('confirmed', 'completed');
9
10 -- IDX-3: Composite on (status, check_in_date) for CTE-1
11 CREATE INDEX IF NOT EXISTS idx_bookings_status_checkin
12 ON bookings (status, check_in_date);

```

Listing 13: Targeted indexes (Milestone 3)

Before and After Results

Table 4: Execution time comparison

Query	Label	Before (ms)	After (ms)	Saved (ms)	% Faster
SQ-1	AGG-2	2.632	0.532	2.100	79.8%
SQ-2	JOIN-2	2.932	1.053	1.879	36.1%
SQ-3	WIN-1	3.811	3.074	0.737	19.3%

Interpretation

SQ-1 delivered the largest gain at 79.8%. The composite index on (`hotel_id`, `rating`) enabled a Merge Join replacing the baseline Hash Join + HashAggregate pipeline. Pre-sorted access eliminates the in-memory hash build entirely.

SQ-2 improved by 36.1% through the same index. The planner switched from HashAggregate over a Hash Join to GroupAggregate over a Merge Join, removing the in-memory hash build across six group-by columns.

SQ-3 improved by 19.3%. The planner retained the Seq Scan on bookings because 373 matching rows out of 450 gives insufficient selectivity on a five-page table to justify index overhead. At production scale, as the bookings table grows, the partial index will be preferred once the planner’s cost estimate crosses the sequential-scan threshold. The Rows Removed by Filter: 77 line confirms the wasted I/O on every current execution.

Stored Procedure

`sp_refresh_hotel_rating_summary` solves the root cause of SQ-2’s cost: the six-column GROUP BY over reviews is re-executed on every RAG ranking request, even though hotel ratings change only when a review is submitted. The procedure materialises this aggregation into a `hotel_rating_summary` table so the RAG layer reads a pre-computed row via primary-key lookup instead of recomputing it live.

```

1 CREATE OR REPLACE PROCEDURE sp_refresh_hotel_rating_summary(
2   p_hotel_id INT DEFAULT NULL

```

```

3 )
4 LANGUAGE plpgsql AS $$
5 DECLARE
6     v_avg    NUMERIC(3,2);
7     v_cnt    INT;
8     v_votes  BIGINT;
9     rec      RECORD;
10 BEGIN
11     IF p_hotel_id IS NOT NULL THEN
12         SELECT ROUND(AVG(rating)::NUMERIC,2), COUNT(*), SUM(helpful_votes)
13         INTO v_avg, v_cnt, v_votes
14         FROM reviews WHERE hotel_id = p_hotel_id;
15
16         INSERT INTO hotel_rating_summary
17             (hotel_id, avg_rating, review_count, total_helpful_votes,
18              last_refreshed_at)
19         VALUES (p_hotel_id, v_avg, v_cnt, v_votes, now())
20         ON CONFLICT (hotel_id) DO UPDATE SET
21             avg_rating          = EXCLUDED.avg_rating,
22             review_count        = EXCLUDED.review_count,
23             total_helpful_votes = EXCLUDED.total_helpful_votes,
24             last_refreshed_at   = EXCLUDED.last_refreshed_at;
25
26         RAISE NOTICE 'Refreshed hotel_id=% | avg=% reviews=% votes=%',
27             p_hotel_id, v_avg, v_cnt, v_votes;
28     ELSE
29         FOR rec IN SELECT DISTINCT hotel_id FROM reviews LOOP
30             SELECT ROUND(AVG(rating)::NUMERIC,2), COUNT(*), SUM(helpful_votes)
31             INTO v_avg, v_cnt, v_votes
32             FROM reviews WHERE hotel_id = rec.hotel_id;
33
34             INSERT INTO hotel_rating_summary
35                 (hotel_id, avg_rating, review_count, total_helpful_votes,
36                  last_refreshed_at)
37             VALUES (rec.hotel_id, v_avg, v_cnt, v_votes, now())
38             ON CONFLICT (hotel_id) DO UPDATE SET
39                 avg_rating          = EXCLUDED.avg_rating,
40                 review_count        = EXCLUDED.review_count,
41                 total_helpful_votes = EXCLUDED.total_helpful_votes,
42                 last_refreshed_at   = EXCLUDED.last_refreshed_at;
43         END LOOP;
44         RAISE NOTICE 'Full summary refresh complete.';
45     END IF;
46 END;
47 $$;

```

Listing 14: Stored procedure definition

Usage:

```

1 CALL sp_refresh_hotel_rating_summary(1);      -- single hotel
2 CALL sp_refresh_hotel_rating_summary(NULL);  -- all hotels

```

RAG Pipeline Implementation

Embedding Generation

Hotel review texts from the `reviews` table are embedded using the `sentence-transformers` library. The model converts each review into a 384-dimensional dense vector that captures semantic meaning. Texts with similar meaning produce vectors that are geometrically close in the embedding space.

Pinecone Vector Store

The 474 review vectors are uploaded to a Pinecone index with cosine similarity as the distance metric. Each vector is stored alongside metadata containing the corresponding `hotel_id` and `review_id`. At query time, the user’s question is embedded and the top- K nearest vectors are retrieved.

Python Query Interface

The Python interface accepts a natural-language string, performs the following steps, and returns a structured response:

1. Embed the question using the same `sentence-transformers` model.
2. Query Pinecone for the top-5 most similar reviews.
3. Extract the `hotel_id` values from the returned metadata.
4. Query PostgreSQL to join `hotels`, `reviews`, and `hotel_rating_summary` on those IDs.
5. Format and return the ranked hotel list with name, price, average rating, and source review citation.

Example

Input	<i>“Best hotel in Miami under \$150 with a pool?”</i>
Output	Courtyard Miami Downtown – \$128/night – Pool & Free WiFi Average rating: 8.4/10 – 12 reviews <i>Source: Review #47 – “Pool area was great and the location was perfect.”</i>

AI Tools Disclosure

In compliance with the project’s AI usage policy, the following tools were used during development:

Tool	Used for	What we changed
Claude (Anthropic)	Drafting SQL query templates, debugging stored procedure	Rewrote queries to use CTEs
GitHub Copilot	Python interface boilerplate	Rewrote all functions to use async

All submitted code was read, understood, and adapted by the team. We take full responsibility for its correctness.

Limitations

1. **Dataset scope.** All 474 hotels are from the Miami area. A user asking about hotels in Nairobi, Zanzibar, or any other city receives no meaningful results. The system does not generalise beyond its training geography.
2. **Synthetic review quality.** Reviews were generated by Faker, not real guests. Real reviews contain slang, emotion, and specific complaints that make embeddings richer and more varied. Faker-generated text is grammatically consistent but lacks the linguistic diversity of authentic user content.
3. **Small dataset.** 1,553 rows is sufficient for a proof of concept but too small for production. With only 474 reviews, semantically similar queries may always retrieve the same top results from Pinecone, reducing the diversity of recommendations.
4. **External dependency on Pinecone.** The Q&A pipeline requires an active internet connection and a live Pinecone API. If the service is unavailable, the system has no fallback. A production deployment would require local vector storage using `pgvector` or a cache layer for recent queries.
5. **No re-ranking step.** Pinecone returns the top- K vectors by cosine similarity. There is no secondary re-ranking step to verify that the retrieved reviews are actually relevant to the question. Low-confidence retrievals are returned without warning.
6. **SQ-3 index not yet used by planner.** The partial index `idx_bookings_hotel_status_partial` is confirmed in the catalogue but the planner still prefers a Seq Scan at 450 rows. The index will be preferred as the bookings table scales past the current five-page threshold.
7. **No authentication or rate limiting.** The Python interface is a command-line script with no user authentication, API key validation, or rate limiting. It is not production-ready for public deployment.

Future Work

1. **Real review data.** Replace Faker-generated reviews with real guest reviews from TripAdvisor or Google Places API. This would make the embedding space richer and retrieval more accurate.
2. **Multi-city expansion.** Ingest hotel datasets covering multiple cities and countries, particularly African markets, which is the intended deployment region.
3. **Replace Pinecone with `pgvector`.** Keeping all data in PostgreSQL using the `pgvector` extension would eliminate the external dependency and reduce latency for small-to-medium scale deployments.
4. **Re-ranking layer.** Add a cross-encoder re-ranking step after Pinecone retrieval to verify that returned reviews are semantically relevant before they are presented to the user.
5. **Confidence threshold.** Return a *"not enough information"* response when the best Pinecone similarity score falls below a defined threshold, rather than returning a potentially wrong recommendation.

6. **Web interface.** Build a web or mobile frontend to replace the command-line interface, enabling real-world user testing.
7. **Multilingual support.** Switch to a multilingual sentence-transformer model (e.g. `multilingual-e5-small`) to support queries in Swahili, French, and other languages spoken in the target market.
8. **Schema additions.** Add a `cities` table to properly normalise the `city` and `state` columns in `hotels`. Introduce a `hotel_amenities` junction table for fully normalised amenity storage alongside the flat `amenities` column retained for embeddings.
9. **Production indexing.** As the bookings table scales, the partial index `idx_bookings_hotel_status` will be activated by the planner. Additionally, partitioning the `bookings` table by year and adding a connection pooler such as PgBouncer would be the appropriate next infrastructure steps.

Conclusion

This project demonstrates that a normalised relational database and a vector store can be combined into a trustworthy hotel recommendation system. The key contribution is not the AI component alone, but the grounding of AI answers in structured, verifiable data. Every recommendation the system produces can be traced back to a specific hotel record and a specific review – a property that distinguishes the system from a generative AI that may hallucinate details.

The project delivered all Track A requirements: a 3NF PostgreSQL schema with five tables and 1,553 rows, ten SQL queries covering all required types, sentence-transformer embeddings stored in Pinecone, a Python Q&A interface with source citations, and EXPLAIN ANALYZE evidence of index improvement across three queries. The stored procedure decouples the most expensive aggregation from the live request path, reducing latency on every RAG response.

The most important technical lesson from this project is that database design decisions – normalisation choices, index strategy, and constraint design – directly determine the quality of an AI system built on top of the database. A poorly designed schema produces poor embeddings, slow queries, and untrustworthy answers. The database is not just storage; it is the foundation that makes AI trustworthy.

Appendix: Setup and Reproduction

Prerequisites

- PostgreSQL 15 or later
- Python 3.10+ with `sentence-transformers`, `pinecone-client`, `psycopg2`, `python-dotenv`
- A Pinecone account (free tier sufficient for development)

Steps

```
1 # 1. Create the database
2 psql -U postgres -c "CREATE DATABASE hotel_rag;"
3
4 # 2. Load schema and data (all INSERT statements are in schema.sql)
```

```
5 psql -U postgres -d hotel_rag -f schema.sql
6
7 # 3. Run queries
8 psql -U postgres -d hotel_rag -f queries.sql
9
10 # 4. Run performance evidence
11 psql -U postgres -d hotel_rag -f performance.sql
12
13 # 5. Set environment variables
14 cp .env.example .env
15 # Edit .env: add PINECONE_API_KEY, PINECONE_ENV, DATABASE_URL
16
17 # 6. Generate embeddings and upload to Pinecone
18 python app/embed.py
19
20 # 7. Run the Q&A interface
21 python app/query.py
```

Listing 15: Reproduction steps

Environment Variables

Variable	Description
DATABASE_URL	PostgreSQL connection string
PINECONE_API_KEY	Pinecone API key
PINECONE_ENV	Pinecone environment region
PINECONE_INDEX	Name of the Pinecone index